

Resume



안녕하세요 이영민입니다.

Email

bbok4yo@gmail.com

Channel

[Github](#)

[Velog](#)

ABOUT ME

- React와 Next.js App Router로 동적 라우팅과 전역 상태 관리를 포함한 화면 아키텍처를 설계합니다.
- TypeScript로 타입을 정의하고 컴포넌트 재사용 구조와 API 호출 시점을 서비스 관점에서 설계합니다.
- 백엔드 데이터 구조와 API 연동을 이해하고 프론트엔드 상태 관리 전략을 함께 설계합니다.
- Git과 PR 기반 협업에서 기획 의도를 확인하고 코드 리뷰에 참여합니다.
- 요구사항 분석부터 SDD 설계까지 개발 흐름을 직접 주도하고, AI를 코드 품질 검증의 보조 도구로 제어합니다.

CAREERS

리얼라인 (단기 인턴십)

기간: 2026.01 ~ 2026.02

- AI 어시스턴트 기반 프론트엔드 생산성 파이프라인 구축 및 POC 검증: 요구사항 분석·시스템 설계·구현·품질 검증 흐름을 직접 설계하고 Claude Code를 보조 도구로 제어해 반복 작업을 자동화했습니다.
- 단기 다중 프로젝트 완수: 1개월 내 전략 보고서 생성기·지도 데이터 시각화·스팟업 웹 3종을 스펙 기반 AI 워크플로우로 구현했습니다.

성과: 개발 흐름 표준화와 품질 엔지니어링

1. AI-Assisted SDD 개발 및 문서화

- 아키텍처 설계: 요구사항을 바탕으로 Claude Code와 상호작용하며 시스템 아키텍처, 데이터 모델링, 컴포넌트 계층을 정의하는 SDD를 신속하게 작성했습니다.
- 문서화 자동화: API 명세 변경과 상태 관리 플로우를 AI로 문서에 실시간 반영해 최신 명세를 바로 확인하도록 했습니다.

2. Custom Skill·Hook 기반 코드 품질 엔지니어링

- 자동화 품질 검증: Custom Skill과 Git Hook(Pre-commit/Pre-push)을 연동하여 커밋 단계에서 AI가 정적 분석·컨벤션 검사를 자동 수행하도록 구성했습니다.
- 지속적 리팩토링: AI 제안을 검증해 보안 취약점 점검·중복 코드 제거·아키텍처 개선 항목의 채택·기각을 직접 판단하고 반영했습니다.

라온피플

기간: 2024.01 ~ 2024.04

성과: QA 기능 테스트 경험을 프론트엔드 방어적 프로그래밍으로 연결

- 기능 테스트와 JIRA 버그 리포트 과정에서 결함 발생 패턴을 분석해 예외 처리·입력 검증 중심의 방어적 프로그래밍 습관을 길렀습니다.
- 테스트 결과 보고서를 작성하고 개발팀에 재현 절차와 개선점을 전달했습니다.
- 관련 기술: JIRA

ACTIVITIES

프론트엔드 개발자 과정

소속/기관: Kakao x goorm

활동 연도: 2024.07~2025.04

- Next.js를 학습하고 실제 프로젝트에 적용해 실전 감각을 키웠습니다.
- 디자이너, PM, 백엔드 개발자 등 다양한 직군과 협업하면서 직군별 의사결정 우선순위 차이를 좁히는 커뮤니케이션 방식을 익혔습니다.

캠퍼스 SW 아카데미 TABA 빅데이터 분석 및 인공지능 처리를 위한 SW융합개발자양성

소속/기관: 과학기술정보통신부

활동 연도: 2023

- 프론트엔드·백엔드 개발 및 Figma 기반의 기획 등 전반적인 개발 협업 과정을 경험했습니다.
- 협업 톨과 개발 전반을 아우르며 실제 프로젝트에서의 기본적인 협업 프로세스를 이해했습니다.
- 과정 수료 후 프로젝트 부분에서 우수상 및 우수교육생으로 선정되었습니다.

EDUCATION

- 단국대학교 컴퓨터공학과 : 2019.03. ~ 2026.02. (졸업)

STACKS

Front-End

- HTML, CSS(SCSS), JS(ES6) & TS
- React.js, Next.js
- tailwind
- Redux Toolkit, react-query, zustand
- D3.js

Collaboration & Tools

- slack

- Figma
- Git, Github

PROJECTS

MindGraph (AI 지식 캡처 & 그래프 시각화)

서비스 개요: ChatGPT·Gemini·Claude·Grok의 답변을 캡처 즉시 의미 단위로 묶어 Cytoscape.js로 시각화하는 Chrome Extension·Next.js 웹앱입니다. 기획·설계·개발·QA 전 구간을 1인 PO로 수행했고 도메인 getmindgraph.com 등록 후 출시 전 1인 프로토타입 단계입니다.

팀원: 1명 (1인 PO·풀스택 개발)

기술 스택: TypeScript, Next.js 16 App Router, React 19, Cytoscape.js, React Flow, Chrome Extension Manifest V3, IndexedDB, Supabase, Claude Code, GitHub Actions

- 1인 PO 환경에서 가설을 빠르게 검증하기 위해 Claude Code 위에 9개 혹·plan·build·qa·ship 4 phase /sprint·4중 SSOT 자동 동기를 설계해 sprint 라이프사이클 반복 작업을 자동화했습니다.
- 매 prompt에 전체 docs가 끌려오는 컨텍스트 과부하를 줄이려고 7부서·9에이전트별 docs를 분리하고 task_type·코드 path glob을 must-read doc에 매핑해 첨부량을 5개 내외로 좁혔습니다.
- 출시 후 가설 검증 인프라를 사전 구축하기 위해 PostHog·Sentry·OpenTelemetry·api/feedback·api/error-log 5개 채널을 코드 베이스에 모았습니다.
- 네트워크 불안정 환경에서 캡처가 유실되지 않도록 IndexedDB 우선 기록과 Pending Ops 큐(MAX_RETRIES=5) 기반 Write-Behind 동기화로 오프라인 CRUD와 온라인 복귀 자동 flush를 구현했습니다.
- TopicNode·QANode 분리 구조의 교차 조회 비용을 해결하기 위해 nodeType·parentId·path·childIds·depth 메타데이터를 가진 UnifiedNode 단일 모델을 설계해 조상 경로 조회를 path 한 번 분해로 처리했습니다.
- Supabase RLS 정책 직접 설계와 GitHub Actions CI/CD·next-intl 한·영 다국어·MV3와 Web origin 화이트리스트 검증 까지 1인이 운영했습니다.

chatGraph (AI 대화 시각화 플랫폼)

서비스 개요: LLM과의 대화를 꼬리물기 형태의 마인드맵 그래프로 시각화해 비선형 사고 흐름을 보존하는 웹 서비스. 4인 팀의 FE 한 명으로 응답 대기 동안의 사용자 흐름과 시각화 라이프사이클을 담당했습니다.

팀원: 4명 (FE 2명, BE 2명)

기술 스택: TypeScript, React, Next.js, D3.js, Zustand, Tailwind CSS, TanStack Query, Vitest

- 새 토픽 생성 시 LLM 응답 대기로 화면이 멈추는 문제를 해결하기 위해 sessionStorage에 프롬프트를 저장하고 temp ID로 즉시 라우팅한 뒤 mutation 성공 시 실제 ID로 교체하는 Optimistic 라우팅을 설계했습니다.
- 사이드바 토픽 클릭 시 초기 로딩 지연을 줄이기 위해 Zustand 스토어에 응답을 프리패치하고 useSuspenseQuery의 initialData로 주입해 라우트 전환 시 빈 화면 없이 데이터가 즉시 그려지게 했습니다.
- 843줄까지 비대해진 use-question-tree 혹을 책임 단위로 분리해 355줄로 축소하고 Feature-First 구조로 도메인·조립·공통 계층을 나눴습니다.
- 거대해진 페이지 컴포넌트를 TopicContentLayout·StandardChatView·OptimisticChatView 뷰 계층으로 나누고 findPathToNode 경로 탐색·토픽 토크 혹에 Vitest 회귀 테스트를 작성했습니다.
- 팀원이 도입한 D3 Force Simulation과 React 리렌더 충돌을, 매 갱신마다 전체 SVG를 재생성하던 방식에서 노드 id 키 기반 enter/update/exit 데이터 조인으로 바꿔 변경 노드만 갱신하고 시뮬레이션을 재사용하도록 개선했습니다.

Fit (사일런트 컨퍼런스 플랫폼)

서비스 개요: 무선 헤드폰을 통해 다중 채널의 발표를 골라 듣는 사일런트 컨퍼런스의 온라인·오프라인 하이브리드 중계 플랫폼

팀원: 5명 (FE 3명, BE 2명)

기술 스택: Vite, React, WebRTC, STOMP, SockJS, Framer Motion, Redux Toolkit, redux-persist, Tailwind CSS

- 현장 저지연 오디오 전송을 위해 RTCPeerConnection API로 발표자·청중 양측 시그널링 클라이언트를 구현하고 STOMP 토픽에서 SDP-ICE 후보를 교환하는 1:N 송수신 흐름을 설계했습니다.
- SDP 협상 전 도착한 ICE 후보가 유실되어 청중이 발표자 음성에 접속하지 못하는 문제를 컴포넌트 ref에 ICE 큐를 두고 setRemoteDescription 직후 일괄 flush하는 버퍼링 로직으로 해결했습니다.
- 모바일 망과 행사장 Wi-Fi의 일시적 단절에 대비해 STOMP 클라이언트에 reconnectDelay 5초와 4초 간격 양방향 heartbeat를 설정하고 자체 TURN 서버를 ICE 후보군에 추가했습니다.
- 시그널링 트래픽과 혼잡도 데이터가 서로의 채널을 점유하지 않도록 혼잡도 전용 STOMP 클라이언트를 별도 구성해 `/sub/session` 토픽만 구독하게 분리했습니다.
- 인증 토큰과 좋아요 세션 목록을 Redux Toolkit으로 관리하고 redux-persist whitelist에 auth만 포함해 새로고침 후 세션 유지와 휘발성 데이터 영속화 방지를 함께 달성했습니다.

xAB (A/B 테스트 기반 SNS)

서비스 개요: 두 가지 선택지에 대한 실시간 투표 및 댓글 토론이 가능한 소셜 네트워크 서비스. Next.js App Router 위에서 프로필·팔로우 도메인과 @modal 인터셉팅 라우트 모달·무한스크롤 피드를 담당했습니다.

팀원: 4명 (FE 4명)

기술 스택: TypeScript, Next.js, React Query, Zustand, Supabase, Tailwind CSS

- 글쓰기 화면 진입 시 피드 위치와 맥락이 끊기는 문제를 App Router의 `@modal` 병렬 슬롯과 `(.)write` 인터셉팅 라우트를 결합한 모달 라우팅으로 해결하고, 직접 접근 시 `app/write` 전체 페이지로 폴백하도록 구성했습니다.
- 프로필·팔로우 도메인을 단독으로 맡아 프로필 페이지(`profile/[id]`)와 팔로워·팔로잉 API·페이지를 구현하고, 팔로우 토글을 서버 액션(`useActionState`)으로 처리한 뒤 `router.refresh()` 로 최신 상태를 반영했습니다.
- 팔로워·팔로잉 목록을 페이지 이동 없이 띄우기 위해 App Router의 `@modal` 병렬 슬롯과 `(.)followers` · `(.)followings` 인터셉팅 라우트를 결합해 모달로 가로채고, 직접 접근 시에는 전체 페이지로 폴백하도록 구성했습니다.
- 메인 피드 초기 로딩 지연을 줄이기 위해 TanStack Query의 `useInfiniteQuery` 와 Intersection Observer 트리거를 결합해 페이지 단위 분할 호출 기반 무한 스크롤 피드를 구현했습니다.
- 팔로우한 유저와 본인 게시글을 묶어 보여주는 피드 API(`api/posts/feed`)를 page-limit-offset 기반 페이지네이션으로 구현하고, A/B 투표·좋아요 API(`api/vote` · `api/like`)를 함께 맡아 피드의 쓰기 경로를 처리했습니다.